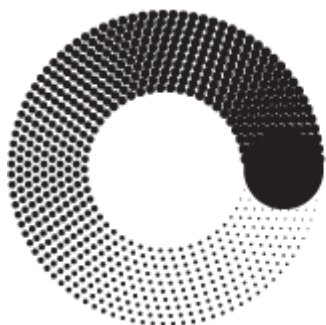


Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования



**МОСКОВСКИЙ
ПОЛИТЕХ**

ОТЧЕТ

по Лабораторной работе 5

по дисциплине: «Технологии компьютерной графики»

Выполнил:	Назаров Т. Д.
Студент Группы:	231-321
Проверили:	Пухова Е.А. Верещагин В.Ю

Москва 2026

1. Цель работы

Изучить методы фильтрации изображений с использованием ядер свертки - реализовать инструмент для применения пользовательских и предустановленных фильтров (размытие по Гауссу, повышение резкости, операторы Прюитта) с возможностью выбора канала и стратегии обработки краев.

2. Постановка задачи

2.1 Фильтрация с использованием ядер

Необходимо добавить возможность работы с фильтрацией, использующей ядра размером 3×3 . В модальном окне должны быть кнопки закрыть, сбросить, применить и чекбокс, включающий предпросмотр. Список предустановленных значений и набор из 9 полей ввода в виде сетки 3×3 . Чекбоксы для выбора каналов применения (R, G, B, A) и выбор стратегии заполнения края (заполнение черным, белым, копирование).

2.2 Предустановленные фильтры

Должны быть реализованы следующие предустановленные фильтры - тождественное отображение, повышение резкости, фильтр Гаусса (3×3), прямоугольное размытие, два ядра из оператора Прюитта (горизонтальное и вертикальное). При выборе варианта из списка 9 полей ввода заполняются соответствующими значениями, пользователь может изменять их на свое усмотрение.

2.3 Обработка краев

Перед применением свертки необходимо обработать края по выбранной стратегии, чтобы размерность изображения не уменьшалась. Для этого используются три стратегии - заполнение черным (значение 0), заполнение белым (значение 255) и копирование крайних пикселей. При обработке больших изображений интерфейс должен оставаться доступным для взаимодействия.

3. Теоретическая часть

3.1 Свертка изображения

Свертка - это математическая операция, при которой каждый пиксель выходного изображения вычисляется как взвешенная сумма соседних пикселей входного изображения с заданными весами (ядром). Формула свертки: $(I * K)(x, y) = \sum_i \sum_j I(x+i, y+j) \times K(i, j)$. Размер ядра обычно составляет 3×3 , но может быть и больше.

3.2 Фильтр Гаусса

Фильтр Гаусса использует ядро, аппроксимирующее двумерную функцию Гаусса. Для ядра 3×3 используется дискретная аппроксимация с суммой весов 16. Ядро имеет вид: $\begin{bmatrix} 1, 2, 1 \\ 2, 4, 2 \\ 1, 2, 1 \end{bmatrix}$ с делителем 16. Этот фильтр обеспечивает плавное размытие с сохранением общей структуры изображения.

3.3 Оператор Прюитта

Оператор Прюитта - это дифференциальный оператор для выделения границ. Он использует два ядра: Prewitt X для вертикальных границ $\begin{bmatrix} -1, 0, 1 \\ -1, 0, 1 \\ -1, 0, 1 \end{bmatrix}$ и Prewitt Y для горизонтальных границ $\begin{bmatrix} -1, -1, -1 \\ 0, 0, 0 \\ 1, 1, 1 \end{bmatrix}$. Вектор градиента вычисляется как $G = \sqrt{G_x^2 + G_y^2}$.

3.4 Стратегии обработки краев

При свертке пиксели на границах изображения не имеют всех необходимых соседей. Для решения этой проблемы используются различные стратегии заполнения - заполнение черным (подстановка нулей), заполнение белым (подстановка 255) и копирование крайних пикселей (дублирование значений с края).

4. Реализация

4.1 Архитектура модуля FiltersTool

```
1138 class FiltersTool {
1139   constructor(app) {
1140     this.app = app;
1141     this.savedImageData = null;
1142     this.updateTimeout = null;
1143     this.init();
1144   }
1145
1146   init() {
1147     this.dialog = document.getElementById('filtersDialog');
1148     this.filtersBtn = document.getElementById('filtersBtn');
1149     this.closeBtn = document.getElementById('closeFiltersDialogBtn');
1150     this.cancelBtn = document.getElementById('cancelFiltersBtn');
1151     this.resetBtn = document.getElementById('resetFiltersBtn');
1152     this.applyBtn = document.getElementById('applyFiltersBtn');
1153     this.presetSelect = document.getElementById('presetFilter');
1154     this.previewCheckbox = document.getElementById('filterPreviewCheckbox');
1155
1156     this.kernelCells = [];
1157     for (let i = 0; i < 3; i++) {
1158       for (let j = 0; j < 3; j++) {
1159         this.kernelCells.push(document.getElementById(`k${i}${j}`));
1160       }
1161     }
1162     this.divisorInput = document.getElementById('kernelDivisor');
1163     this.channelRed = document.getElementById('channelRed');
1164     this.channelGreen = document.getElementById('channelGreen');
1165     this.channelBlue = document.getElementById('channelBlue');
1166     this.channelAlpha = document.getElementById('channelAlpha');
1167     this.edgeStrategy = document.getElementById('edgeStrategy');
1168
1169     this.filtersBtn.onclick = () => this.openDialog();
1170     this.closeBtn.onclick = () => this.dialog.close();
1171     this.cancelBtn.onclick = () => this.cancel();
1172     this.resetBtn.onclick = () => this.reset();
1173     this.applyBtn.onclick = () => this.apply();
1174
1175     this.presetSelect.onChange = () => this.loadPreset();
1176     this.previewCheckbox.onChange = () => this.togglePreview();
1177
1178     const updateKernel = () => this.scheduleUpdate();
1179     this.kernelCells.forEach(cell => cell.oninput = updateKernel);
1180     this.divisorInput.oninput = updateKernel;
1181     this.edgeStrategy.onChange = () => this.scheduleUpdate();
1182     [this.channelRed, this.channelGreen, this.channelBlue, this.channelAlpha].forEach(ch => {
1183       ch.onChange = () => this.scheduleUpdate();
1184     });
1185   }
1186 }
```

4.2 Реализация свертки с обработкой краев

```

1263     applyFilterToImage() {
1264         if (!this.app.originalImageData) return;
1265         const { kernel, divisor } = this.getKernel();
1266         const strategy = this.edgeStrategy.value;
1267         const applyToRed = this.channelRed.checked;
1268         const applyToGreen = this.channelGreen.checked;
1269         const applyToBlue = this.channelBlue.checked;
1270         const applyToAlpha = this.channelAlpha.checked;
1271         const src = this.app.originalImageData.data;
1272         const w = this.app.originalImageData.width;
1273         const h = this.app.originalImageData.height;
1274         const newData = new Uint8ClampedArray(src.length);
1275         for (let y = 0; y < h; y++) {
1276             for (let x = 0; x < w; x++) {
1277                 let sumR = 0, sumG = 0, sumB = 0, sumA = 0;
1278                 for (let ky = -1; ky <= 1; ky++) {
1279                     for (let kx = -1; kx <= 1; kx++) {
1280                         const weight = kernel[ky+1][kx+1];
1281                         const ix = x + kx;
1282                         const iy = y + ky;
1283                         sumR += this.getEdgeValue(strategy, src, ix, iy, w, h, 0) * weight;
1284                         sumG += this.getEdgeValue(strategy, src, ix, iy, w, h, 1) * weight;
1285                         sumB += this.getEdgeValue(strategy, src, ix, iy, w, h, 2) * weight;
1286                         sumA += this.getEdgeValue(strategy, src, ix, iy, w, h, 3) * weight;
1287                     }
1288                 }
1289                 const idx = (y * w + x) * 4;
1290                 if (applyToRed) {
1291                     let val = sumR / divisor;
1292                     newData[idx] = Math.min(255, Math.max(0, Math.round(val)));
1293                 } else { newData[idx] = src[idx]; }
1294                 if (applyToGreen) {
1295                     let val = sumG / divisor;
1296                     newData[idx+1] = Math.min(255, Math.max(0, Math.round(val)));
1297                 } else { newData[idx+1] = src[idx+1]; }
1298                 if (applyToBlue) {
1299                     let val = sumB / divisor;
1300                     newData[idx+2] = Math.min(255, Math.max(0, Math.round(val)));
1301                 } else { newData[idx+2] = src[idx+2]; }
1302                 if (applyToAlpha) {
1303                     let val = sumA / divisor;
1304                     newData[idx+3] = Math.min(255, Math.max(0, Math.round(val)));
1305                 } else { newData[idx+3] = src[idx+3]; }
1306             }
1307         }
1308         const newImageData = new ImageData(w, h);
1309         newImageData.data.set(newData);
1310         this.app.currentImageData = newImageData;
1311         this.app.renderCanvas();
1312     }
1313 }

```

4.3 Обработка краев по выбранной стратегии

```

1248     getEdgeValue(strategy, src, x, y, w, h, channel) {
1249         if (x >= 0 && x < w && y >= 0 && y < h) {
1250             return src[(y * w + x) * 4 + channel];
1251         }
1252         switch(strategy) {
1253             case 'black': return 0;
1254             case 'white': return 255;
1255             case 'extend':
1256                 const ex = Math.min(Math.max(x, 0), w - 1);
1257                 const ey = Math.min(Math.max(y, 0), h - 1);
1258                 return src[(ey * w + ex) * 4 + channel];
1259             default: return 0;
1260         }
1261     }

```

4.4 Debounce для предпросмотра

```
1186     scheduleUpdate() {  
1187         if (this.updateTimeout) clearTimeout(this.updateTimeout);  
1188         this.updateTimeout = setTimeout(() => {  
1189             if (this.previewCheckbox.checked) {  
1190                 this.applyFilterToImage();  
1191             }  
1192             this.updateTimeout = null;  
1193         }, 50);  
    }
```

5. Интерфейс пользователя

Диалоговое окно содержит выпадающий список предустановленных фильтров, сетку полей ввода 3×3 для редактирования ядра, поле ввода делителя, чекбоксы выбора каналов (R, G, B, A), выпадающий список для выбора стратегии обработки краев, чекбокс предпросмотра и кнопки СБРОС, ОТМЕНА, ПРИМЕНИТЬ.

ФИЛЬТРАЦИЯ ИЗОБРАЖЕНИЯ ✕

ПРЕДУСТАНОВКИ: ТОЖДЕСТВЕННЫЙ ▾

ЯДРО СВЁРТКИ (3×3)

0	0	0
0	1	0
0	0	0

ДЕЛИТЕЛЬ: 1

ПРИМЕНИТЬ К:

☒ R ☒ G ☒ B ☐ A

ОБРАБОТКА КРАЁВ: ЗАПОЛНЕНИЕ ЧЁРНЫМ (0) ▾

☒ ПРЕДПРОСМОТР (ДО/ПОСЛЕ)

⌂ СБРОС ОТМЕНА ПРИМЕНИТЬ

6. Заключение

В ходе выполнения лабораторной работы реализованы шесть предустановленных фильтров - тождественный, повышение резкости, размытие по Гауссу, прямоугольное размытие, операторы Прюитта X и Y. Реализована возможность редактирования ядра 3×3 пользователем, выбор каналов для применения фильтра, три стратегии обработки краев (черный, белый, копирование). Для плавной работы добавлен debounce задержкой 50 мс.